

AI ASSISTED CODING – 8.3

NAME : Pothunuri AKSHITHA

ROLL NO : 2403A510C2

BATCH : 05

DEPT : CSE

Task – 01

Prompt:

Write a python code for valid email id where the it must contain @ and (.) character ,should not end with special characters , and should not contain multiple @.

Code generated:

```
python.py X
python.py > ...
1  def is_valid_email(email):
2      """
3      Validates an email address based on the following criteria:
4      - Must contain @ character
5      - Must contain . character
6      - Should not end with special characters
7      - Should not contain multiple @ characters
8      """
9
10     # Check if email is empty or None
11     if not email:
12         return False
13
14     # Check if email contains @ character
15     if '@' not in email:
16         return False
17
18     # Check if email contains . character
19     if '.' not in email:
20         return False
21
22     # Check if email contains multiple @ characters
23     if email.count('@') > 1:
24         return False
25
26     # Check if email ends with special characters
27     # Define special characters (excluding alphanumeric and dot)
28     special_chars = "!@#%&*()_+=[{}|;':\".,/<>?`~"
29     if email[-1] in special_chars:
30         return False
31
```

```
python.py X
python.py > test_email_validation
1  def is_valid_email(email):
2
3  |   return True
4
5  |
6  |
7  |
8  |
9  |
10 |
11 |
12 |
13 |
14 |
15 |
16 |
17 |
18 |
19 |
20 |
21 |
22 |
23 |
24 |
25 |
26 |
27 |
28 |
29 |
30 |
31 |
32 |
33 |
34 |
35 |
36 |
37 |
38 |
39 |
40 |
41 |
42 |
43 |
44 |
45 |
46 |
47 |
48 |
49 |
50 |
51 |
52 |
53 |
54 |
55 |
56 |
57 |
58 |
59 |
60 |
61 |
62 |
63 |
64 |
65 |
66 |
67 |
68 |
69 |
70 |
71 |
72 |
73 |
74 |
75 |
76 |
77 |
78 |
79 |
80 |
81 |
82 |
83 |
84 |
85 |
86 |
87 |
88 |
89 |
90 |
91 |
92 |
93 |
94 |
95 |
96 |
97 |
98 |
99 |
100|

# Test the function with various email examples
def test_email_validation():
    """Test function to demonstrate email validation"""
    test_emails = [
        "user@example.com",      # Valid
        "test.email@domain.org", # Valid
        "user123@test.co.uk",    # Valid
        "invalid-email",         # Invalid - no @
        "user@domain",           # Invalid - no .
        "user@domain.com",       # Invalid - multiple @
        "user@domain.com!",      # Invalid - ends with special char
        "user@domain.com@",      # Invalid - ends with @
        "user@domain.com.",      # Invalid - ends with .
        "",                       # Invalid - empty
        "user@domain@com",       # Invalid - multiple @
    ]

    print("Email Validation Test Results:")
    print("=" * 40)

    for email in test_emails:
        result = is_valid_email(email)
        status = "✓ VALID" if result else "X INVALID"
        print(f"{email:<25} -> {status}")

# Example usage
if __name__ == "__main__":
    # Run the test
    test_email_validation()

    print("\n" + "=" * 40)
    print("Interactive Email Validation")
    print("=" * 40)

    # Interactive testing
    while True:
        email = input("\nEnter an email to validate (or 'quit' to exit): ").strip()

        if email.lower() == 'quit':
            break

        if is_valid_email(email):
            print(f"✓ '{email}' is a valid email address!")
        else:
            print(f"X '{email}' is NOT a valid email address!")
```

```
print("Email Validation Test Results:")
print("=" * 40)
| Ctrl+L to chat, Ctrl+K to generate
for email in test_emails:
    result = is_valid_email(email)
    status = "✓ VALID" if result else "X INVALID"
    print(f"{email:<25} -> {status}")

# Example usage
if __name__ == "__main__":
    # Run the test
    test_email_validation()

    print("\n" + "=" * 40)
    print("Interactive Email Validation")
    print("=" * 40)

    # Interactive testing
    while True:
        email = input("\nEnter an email to validate (or 'quit' to exit): ").strip()

        if email.lower() == 'quit':
            break

        if is_valid_email(email):
            print(f"✓ '{email}' is a valid email address!")
        else:
            print(f"X '{email}' is NOT a valid email address!")
```

Output:

```
Problems Output Debug Console Terminal Ports
PS C:\Users\nadhi\OneDrive\Desktop\AI> & C:/Users/nadhi/anaconda3/python.exe c:/Users/nadhi/OneDrive/Desktop/AI/python.py
Email Validation Test Results:
=====
user@example.com      -> ✓ VALID
test.email@domain.org -> ✓ VALID
user123@test.co.uk    -> ✓ VALID
invalid-email         -> ✗ INVALID
user@domain           -> ✗ INVALID
user@domain.com       -> ✗ INVALID
user@domain.com!      -> ✗ INVALID
user@domain.com@      -> ✗ INVALID
user@domain.com.      -> ✗ INVALID
user@domain@com       -> ✗ INVALID
=====
Interactive Email Validation
=====
Enter an email to validate (or 'quit' to exit): nadhiya
✗ 'nadhiya' is NOT a valid email address!

Enter an email to validate (or 'quit' to exit): nadhiya@gmail.com
✓ 'nadhiya@gmail.com' is a valid email address!

Enter an email to validate (or 'quit' to exit): exit
✗ 'exit' is NOT a valid email address!

Enter an email to validate (or 'quit' to exit):
```

Observation:

1. **Simple Logic Flow**: The function uses straightforward if-statements to check each requirement sequentially, making it easy to understand and debug.
2. **No External Dependencies**: The code uses only built-in Python functions (`count()`, `in`, `not`) without any imports, keeping it lightweight and portable.
3. **Comprehensive Test Coverage**: The test function includes both valid and invalid email examples covering all validation rules, making it easy to verify the function works correctly.

Task – 02

Prompt:

write a python code to test cases for assigning grades) where: 90-100: A, 80-89: B, 70-

79: C, 60-69: D, <60: F

- Include boundary values and invalid inputs (e.g., -5, 105, "eighty").

Code generated:

```
python.py > ...
1 def assign_grade(score):
2     """Assign grade based on score: 90-100: A, 80-89: B, 70-79: C, 60-69: D, <60: F"""
3     if not isinstance(score, (int, float)):
4         return "Invalid input: Score must be a number"
5
6     if score < 0 or score > 100:
7         return "Invalid input: Score must be between 0 and 100"
8
9     if score >= 90:
10        return "A"
11    elif score >= 80:
12        return "B"
13    elif score >= 70:
14        return "C"
15    elif score >= 60:
16        return "D"
17    else:
18        return "F"
19
20
21 def test_grade_assignment():
22     """Test cases for grade assignment function"""
23     test_cases = [
24         # Boundary values
25         (100, "A"), (90, "A"), (89, "B"), (80, "B"),
26         (79, "C"), (70, "C"), (69, "D"), (60, "D"), (59, "F"), (0, "F"),
27
28         # Invalid inputs
29         (-5, "Invalid input: Score must be between 0 and 100"),
30         (105, "Invalid input: Score must be between 0 and 100"),
31         ("eighty", "Invalid input: Score must be a number"),
32         ("90", "Invalid input: Score must be a number"),
33         (None, "Invalid input: Score must be a number"),
34
35         # Normal cases
36         (95, "A"), (85, "B"), (75, "C"), (65, "D"), (55, "F")
37     ]
38
39     print("Testing Grade Assignment Function")
40     print("=" * 60)
41
42     passed = 0
```

```

python.py / ...
21 def test_grade_assignment():
22     (100, "A"), (90, "A"), (89, "B"), (80, "B"),
23     (79, "C"), (70, "C"), (69, "D"), (60, "D"), (59, "F"), (0, "F"),
24
25     # Invalid inputs
26     (-5, "Invalid input: Score must be between 0 and 100"),
27     (105, "Invalid input: Score must be between 0 and 100"),
28     ("eighty", "Invalid input: Score must be a number"),
29     ("90", "Invalid input: Score must be a number"),
30     (None, "Invalid input: Score must be a number"),
31
32     # Normal cases
33     (95, "A"), (85, "B"), (75, "C"), (65, "D"), (55, "F")
34 ]
35
36 print("Testing Grade Assignment Function")
37 print("=" * 60)
38
39 passed = 0
40 total = len(test_cases)
41
42 for score, expected in test_cases:
43     result = assign_grade(score)
44     status = "PASS" if result == expected else "FAIL"
45     if status == "PASS":
46         passed += 1
47
48     print(f"Score: {score!r}>8 | Expected: {expected!r}>30 | Got: {result!r}>30 | {status}")
49
50 print("=" * 60)
51 print(f"Test Results: {passed}/{total} tests passed")
52 return passed == total
53
54 if __name__ == "__main__":
55     test_grade_assignment()
56
57
58
59
60

```

Output:

```

45     for score, expected in test_cases:
46         result = assign_grade(score)

```

Problems Output Debug Console Terminal Ports

Testing Grade Assignment Function

```

=====
Score: 100 | Expected: 'A' | Got: 'A' | PASS
Score: 90 | Expected: 'A' | Got: 'A' | PASS
Score: 89 | Expected: 'B' | Got: 'B' | PASS
Score: 80 | Expected: 'B' | Got: 'B' | PASS
Score: 79 | Expected: 'C' | Got: 'C' | PASS
Score: 70 | Expected: 'C' | Got: 'C' | PASS
Score: 69 | Expected: 'D' | Got: 'D' | PASS
Score: 60 | Expected: 'D' | Got: 'D' | PASS
Score: 59 | Expected: 'F' | Got: 'F' | PASS
Score: 0 | Expected: 'F' | Got: 'F' | PASS
Score: -5 | Expected: 'Invalid input: Score must be between 0 and 100' | Got: 'Invalid input: Score must be between 0 and 100' | PASS
Score: 105 | Expected: 'Invalid input: Score must be between 0 and 100' | Got: 'Invalid input: Score must be between 0 and 100' | PASS
Score: 'eighty' | Expected: 'Invalid input: Score must be a number' | Got: 'Invalid input: Score must be a number' | PASS
Score: '90' | Expected: 'Invalid input: Score must be a number' | Got: 'Invalid input: Score must be a number' | PASS
Score: None | Expected: 'Invalid input: Score must be a number' | Got: 'Invalid input: Score must be a number' | PASS
Score: 95 | Expected: 'A' | Got: 'A' | PASS
Score: 85 | Expected: 'B' | Got: 'B' | PASS
Score: 75 | Expected: 'C' | Got: 'C' | PASS
Score: 65 | Expected: 'D' | Got: 'D' | PASS
Score: 55 | Expected: 'F' | Got: 'F' | PASS
=====
Test Results: 20/20 tests passed
PS C:\Users\nadhi\OneDrive\Desktop\AI>

```

Observation:

The code effectively tests all boundary conditions (90, 89, 80, 79, etc.) and invalid inputs (negative numbers, over 100, non-numeric strings) with clear pass/fail reporting, making it a comprehensive test suite for grade assignment logic.

The formatting improvements you made (using !r for repr() and adjusting column widths) enhance readability by properly displaying string inputs with quotes and aligning the output columns better for visual clarity.

Task -03

Prompt:

write a python code for a sentence palindrome where test cases for is_sentence_palindrome(sentence) (ignores case, spaces, and punctuation).

- Example:

"A man a plan a canal Panama" → True.

Code generated:

```
python.py X
python.py > ...
1  import re
2
3  def is_sentence_palindrome(sentence):
4      """
5      Check if a sentence is a palindrome, ignoring case, spaces, and punctuation.
6
7      Args:
8      |   sentence (str): The sentence to check
9
10     Returns:
11     |   bool: True if the sentence is a palindrome, False otherwise
12     """
13     # Remove all non-alphabetic characters and convert to lowercase
14     cleaned = re.sub(r'[^\w]', '', sentence.lower())
15
16     # Check if the cleaned string is equal to its reverse
17     return cleaned == cleaned[::-1]
18
19 def test_is_sentence_palindrome():
20     """Test cases for is_sentence_palindrome function"""
21
22     # Test case 1: Classic palindrome
23     assert is_sentence_palindrome("A man a plan a canal Panama") == True
24
25     # Test case 2: Simple palindrome
26     assert is_sentence_palindrome("racecar") == True
27
28     # Test case 3: Palindrome with punctuation
29     assert is_sentence_palindrome("Was it a car or a cat I saw?") == True
30
31     # Test case 4: Not a palindrome
32     assert is_sentence_palindrome("Hello world") == False
33
34     # Test case 5: Empty string
35     assert is_sentence_palindrome("") == True
36
37     # Test case 6: Single character
38     assert is_sentence_palindrome("a") == True
39
40     # Test case 7: Palindrome with numbers and special characters
41     assert is_sentence_palindrome("A1B2C3C2B1A") == True
42
43     # Test case 8: Case insensitive test
44     assert is_sentence_palindrome("Madam") == True
```

```
python.py > ...
19 def test_is_sentence_palindrome():
20     assert is_sentence_palindrome("A man a plan a canal Panama") == True
21
22
23     # Test case 2: Simple palindrome
24     assert is_sentence_palindrome("racecar") == True
25
26     # Test case 3: Palindrome with punctuation
27     assert is_sentence_palindrome("Was it a car or a cat I saw?") == True
28
29     # Test case 4: Not a palindrome
30     assert is_sentence_palindrome("Hello world") == False
31
32     # Test case 5: Empty string
33     assert is_sentence_palindrome("") == True
34
35     # Test case 6: Single character
36     assert is_sentence_palindrome("a") == True
37
38     # Test case 7: Palindrome with numbers and special characters
39     assert is_sentence_palindrome("A1B2C3C2B1A") == True
40
41     # Test case 8: Case insensitive test
42     assert is_sentence_palindrome("Madam") == True
43
44     # Test case 9: Complex palindrome with mixed punctuation
45     assert is_sentence_palindrome("A Santa at NASA!") == True
46
47     # Test case 10: Non-palindrome with similar characters
48     assert is_sentence_palindrome("Palindrome") == False
49
50     print("All test cases passed!")
51
52
53
54 if __name__ == "__main__":
55     # Run the test cases
56     test_is_sentence_palindrome()
57
58     # Example usage
59     test_sentence = "A man a plan a canal Panama"
60     result = is_sentence_palindrome(test_sentence)
61     print(f"'{test_sentence}' is a palindrome: {result}")
```

Output:

```
Score: 55 | Expected: 'F' | Got: 'F' | PASS
=====
Test Results: 20/20 tests passed
PS C:\Users\nadhi\OneDrive\Desktop\AI> & C:/Users/nadhi/anaconda3/python.exe c:/Users/nadhi/OneDrive/Desktop/AI/python.py
All test cases passed!
'A man a plan a canal Panama' is a palindrome: True
PS C:\Users\nadhi\OneDrive\Desktop\AI> 
```

Observation:

1. *Efficient cleaning approach:* The regex `re.sub(r'^a-zA-Z|', '', sentence.lower())` removes all non-alphabetic characters in one operation, making it more efficient than multiple string operations.
2. *Pythonic palindrome check:* Using `cleaned == cleaned[::-1]` leverages Python's slice notation to reverse the string, which is both readable and performant compared

Task – 04

Prompt:

write a python code to nerate test cases for a ShoppingCart class
(add_item,remove_item, total) where code should contain
Add_item(name,orice),Remove_item(name),total_cost().

Code generated:

```
python.py > ...
class ShoppingCart:
    def __init__(self):
        self.items = {}

    def add_item(self, name, price):
        if not isinstance(name, str) or not isinstance(price, (int, float)) or price < 0:
            return "Invalid input"
        self.items[name] = self.items.get(name, 0) + price
        return f"Added {name} for {price}"

    def remove_item(self, name):
        if name in self.items:
            del self.items[name]
            return f"Removed {name}"
        return "Item not found"

    def total_cost(self):
        return sum(self.items.values())

def test_shopping_cart():
    cart = ShoppingCart()
    test_cases = [
        ("add", "Apple", 50, "Added Apple for 50"),
        ("add", "Banana", 30, "Added Banana for 30"),
        ("add", "Apple", 50, "Added Apple for 50"),
        ("remove", "Banana", None, "Removed Banana"),
        ("remove", "Mango", None, "Item not found"),
        ("add", "Orange", -10, "Invalid input"),
        ("add", 123, 40, "Invalid input"),
    ]
    print("Testing ShoppingCart")
```



```
python.py X
python.py > ...
21 def test_shopping_cart():
22     test_cases = [
23         ("add", "Apple", 50, "Added Apple for 50"),
24         ("remove", "Banana", None, "Removed Banana"),
25         ("remove", "Mango", None, "Item not found"),
26         ("add", "Orange", -10, "Invalid input"),
27         ("add", 123, 40, "Invalid input"),
28     ]
29
30     print("Testing ShoppingCart")
31     print("=" * 50)
32     for action, name, price, expected in test_cases:
33         if action == "add":
34             result = cart.add_item(name, price)
35         elif action == "remove":
36             result = cart.remove_item(name)
37         print(f"{action.title()} {name}: Expected={expected}, Got={result},",
38               "PASS" if result == expected else "FAIL")
39
40     # Final check for total cost
41     total = cart.total_cost()
42     print("=" * 50)
43     print(f"Total cost Expected=100, Got={total}, PASS" if total == 100 else "FAIL")
44
45 if __name__ == "__main__":
46     test_shopping_cart()
47
48 Ctrl+L to chat, Ctrl+K to generate
```

Output:

```
OK
PS C:\Users\nadhi\OneDrive\Desktop\AI> & C:/Users/nadhi/anaconda3/python.exe c:/Users/nadhi/OneDrive/Desktop/AI/python.py
Testing ShoppingCart
=====
Add 'Apple': Expected='Added Apple for 50', Got='Added Apple for 50', PASS
Add 'Banana': Expected='Added Banana for 30', Got='Added Banana for 30', PASS
Add 'Apple': Expected='Added Apple for 50', Got='Added Apple for 50', PASS
Remove 'Banana': Expected='Removed Banana', Got='Removed Banana', PASS
Remove 'Mango': Expected='Item not found', Got='Item not found', PASS
Add 'Orange': Expected='Invalid input', Got='Invalid input', PASS
Add 123: Expected='Invalid input', Got='Invalid input', PASS
=====
Total cost Expected=100, Got=100 PASS
PS C:\Users\nadhi\OneDrive\Desktop\AI>
```

Observation:

The code correctly tests all functionalities of the **ShoppingCart** class, including valid/invalid additions, removals, and total cost calculation. It handles edge cases well and produces clear PASS/FAIL outputs for each test case.

Task – 05:

Prompt:

write a python code to test cases for `convert_date_format(date_str)` to switch from "YYYY-MM-DD" to "DD-MM-YYYY".

Example: "2023-10-15" → "15-10-2023" make sure user should give input.

Code generated

```
python.py ^ ...
1  def convert_date_format(date_str):
2      """
3      Convert date from YYYY-MM-DD format to DD-MM-YYYY format
4      """
5      try:
6          # Split the date string by '-'
7          parts = date_str.split('-')
8
9          # Check if we have exactly 3 parts
10         if len(parts) != 3:
11             raise ValueError("Invalid date format")
12
13         year, month, day = parts
14
15         # Validate that each part has correct length
16         if len(year) != 4 or len(month) != 2 or len(day) != 2:
17             raise ValueError("Invalid date format")
18
19         # Return in DD-MM-YYYY format
20         return f"{day}-{month}-{year}"
21
22     except Exception as e:
23         return f"Error: {str(e)}"
24
25 def test_convert_date_format():
26     """Test cases for convert_date_format function"""
27     test_cases = [
28         ("2023-10-15", "15-10-2023"),
29         ("2024-01-01", "01-01-2024"),
30         ("2023-12-31", "31-12-2023"),
31         ("2024-02-29", "29-02-2024"), # Leap year
32         ("2023-06-05", "05-06-2023"),
33         ("invalid-date", "Error: Invalid date format").
```

```
python.py > ...
25 def test_convert_date_format():
26     """Test cases for convert_date_format function"""
27     test_cases = [
28         ("2023-10-15", "15-10-2023"),
29         ("2024-01-01", "01-01-2024"),
30         ("2023-12-31", "31-12-2023"),
31         ("2024-02-29", "29-02-2024"), # Leap year
32         ("2023-06-05", "05-06-2023"),
33         ("invalid-date", "Error: Invalid date format"),
34         ("2023-13-01", "Error: Invalid date format"), # Invalid month
35         ("2023-01-32", "Error: Invalid date format"), # Invalid day
36     ]
37
38     print("Testing convert_date_format function:")
39     print("=" * 50)
40
41     for i, (input_date, expected) in enumerate(test_cases, 1):
42         result = convert_date_format(input_date)
43         status = "✓ PASS" if result == expected else "X FAIL"
44         print(f"Test {i}: {input_date} → {result} [{status}]")
45         if result != expected:
46             print(f"    Expected: {expected}")
47
48 def main():
49     """Main function with user input"""
50     print("Date Format Converter")
51     print("Convert from YYYY-MM-DD to DD-MM-YYYY")
52     print("=" * 40)
53
54     # Run test cases
55     test_convert_date_format()
56
57     print("\n" + "=" * 40)
```

Problems Output Debug Console Terminal Ports

```
python.py X
python.py > ...
46         print(f"    Expected: {expected}")
47
48 def main():
49     """Main function with user input"""
50     print("Date Format Converter")
51     print("Convert from YYYY-MM-DD to DD-MM-YYYY")
52     print("=" * 40)
53
54     # Run test cases
55     test_convert_date_format()
56
57     print("\n" + "=" * 40)
58     print("User Input Testing:")
59
60     while True:
61         user_input = input("\nEnter a date (YYYY-MM-DD) or 'quit' to exit: ")
62
63         if user_input.lower() == 'quit':
64             print("Goodbye!")
65             break
66
67         result = convert_date_format(user_input)
68         print(f"Converted: {result}")
69
70 if __name__ == "__main__":
71     main()
72
```

Output:

```

4.3...
docx
pdf
Total cost Expected=100, Got=100 PASS
PS C:\Users\nadhi\OneDrive\Desktop\AI> & C:/Users/nadhi/anaconda3/python.exe c:/Users/nadhi/OneDrive/Desktop/AI/python.py
Date Format Converter
Convert from YYYY-MM-DD to DD-MM-YYYY
=====
Testing convert_date_format function:
=====
Test 1: 2023-10-15 → 15-10-2023 [✓ PASS]
Test 2: 2024-01-01 → 01-01-2024 [✓ PASS]
Test 3: 2023-12-31 → 31-12-2023 [✓ PASS]
Test 4: 2024-02-29 → 29-02-2024 [✓ PASS]
Test 5: 2023-06-05 → 05-06-2023 [✓ PASS]
Test 6: invalid-date → Error: Invalid date format [✓ PASS]
Test 7: 2023-13-01 → 01-13-2023 [X FAIL]
Expected: Error: Invalid date format
Test 8: 2023-01-32 → 32-01-2023 [X FAIL]
Expected: Error: Invalid date format
=====
User Input Testing:

Enter a date (YYYY-MM-DD) or 'quit' to exit: 2015-01-22
Converted: 22-01-2015

Enter a date (YYYY-MM-DD) or 'quit' to exit: 2023-03-12
Converted: 12-03-2023

Enter a date (YYYY-MM-DD) or 'quit' to exit: exit
Converted: Error: Invalid date format

Enter a date (YYYY-MM-DD) or 'quit' to exit: 

```

Observation:

The code successfully converts dates from **YYYY-MM-DD** to **DD-MM-YYYY** format and includes both automated test cases and interactive user input.

However, it only checks string structure, not actual calendar validity (e.g., invalid months/days may pass if formatted correctly).